ПОЛНЫЙ КОД

```c
#include <stdio.h>
#include <stdlib.h>
#include <sys/wait.h>
#include <unistd.h>
#include <fcntl.h>
#include <stdint.h>

int global_var = 100;


uintptr_t get_page(uintptr_t virtual_addr) {
    int fd = open("/proc/self/pagemap", O_RDONLY);
    if (fd < 0) {
        perror("open pagemap (run with sudo if needed)");
        return 0;
    }

    uint64_t value;
    long page_size = sysconf(_SC_PAGESIZE);
    off_t offset = (virtual_addr / page_size) * sizeof(value);

    if (pread(fd, &value, sizeof(value), offset) != sizeof(value))
{
        close(fd);
        return 0;
    }
    close(fd);


    if (!(value & (1ULL << 63))) {
        return 0;
    }

    uint64_t pfn = value & ((1ULL << 55) - 1);
    return pfn;
}

void print_ptrs(const char* label, int* g, int* l) {
    printf("--- %s ---\n", label);
    uintptr_t phys_g = get_page((uintptr_t)g);
    uintptr_t phys_l = get_page((uintptr_t)l);

    printf("Virt: G=%p, L=%p\n", (void*)g, (void*)l);
    printf("Phys PFN: G=0x%lx, L=0x%lx\n", phys_g, phys_l);
    printf("Val : G=%d, L=%d\n", *g, *l);
    printf("\n");
}

int main() {
    int local_var = 10;
```

```
    pid_t pid;

    print_ptrs("До fork()", &global_var, &local_var);

    pid = fork();

    if (pid < 0) {
        perror("fork failed");
        exit(1);
    }
    else if (pid == 0) {
        printf("--- В дочернем процессе (PID: %d) ---\n",
getpid());
        print_ptrs("Сразу после fork (чтение)", &global_var,
&local_var);

        printf("Изменяем переменные в дочернем процессе...\n\n");
        global_var = 200;
        local_var = 20;

        print_ptrs("После изменения (запись)", &global_var,
&local_var);

        exit(5);
    }
    else {
        sleep(2);
        printf("--- В родительском процессе (PID: %d) ---\n",
getpid());
        print_ptrs("После того как ребенок изменил свои копии",
&global_var, &local_var);

        printf("Родитель засыпает (проверьте ps/maps)...\n");
        sleep(30);

        int status;
        waitpid(pid, &status, 0);

        if (WIFEXITED(status)) {
            printf("\nДочерний процесс завершен. Код: %d\n",
WEXITSTATUS(status));
        }
    }

    return 0;
}
```

ПОЯСНЕНИЕ

```
// Принимает виртуальный адрес, возвращает номер физической страницы (PFN)
uintptr_t get_page(uintptr_t virtual_addr) {

    // Открываем /proc/self/pagemap — специальный файл ядра.
    // Для каждой виртуальной страницы там хранится 8-байтовая запись с PFN и флагами.
    // O_RDONLY — только чтение, писать туда нельзя.
    int fd = open("/proc/self/pagemap", O_RDONLY);

    if (fd < 0) {
        // Если не удалось открыть — скорее всего нет прав (нужен sudo или
CAP_SYS_ADMIN)
        perror("open pagemap (run with sudo if needed)");
        return 0;
    }

    uint64_t value;  // сюда прочитаем 8-байтовую запись из pagemap

    // sysconf(_SC_PAGESIZE) возвращает размер страницы — обычно 4096 байт (4 КБ)
    long page_size = sysconf(_SC_PAGESIZE);

    // Вычисляем смещение внутри файла pagemap.
    // Логика: pagemap — это массив 8-байтовых записей, по одной на каждую страницу.
    // Номер страницы = virtual_addr / page_size (целочисленное деление, отбрасывает
остаток)
    // Смещение в файле = номер_страницы * 8 (sizeof(uint64_t))
    // Пример: addr=0x601020, page_size=4096 → страница №0x601 → offset=0x601*8
    off_t offset = (virtual_addr / page_size) * sizeof(value);

    // pread — читает из файла по конкретному смещению, не меняя позицию файлового
дескриптора.
    // Аргументы: fd, куда_читать, сколько_байт, с_какого_смещения
    // Если прочитали не 8 байт — что-то пошло не так (страница не замаплена и т.д.)
    if (pread(fd, &value, sizeof(value), offset) != sizeof(value)) {
        close(fd);
        return 0;
    }

    close(fd);  // всегда закрываем дескриптор

    // Проверяем бит 63 записи pagemap — он означает "страница сейчас в RAM"
    // (1ULL << 63) создаёт маску: 1000...000 (63 нуля)
    // Если бит 63 == 0 — страница выгружена в swap или ещё не аллоцирована → PFN
недоступен
    if (!(value & (1ULL << 63))) {
        return 0;
    }

    // Извлекаем PFN из битов 0–54.
    // (1ULL << 55) - 1 создаёт маску: 0000...0111...1 (55 единиц)
    // Побитовое AND оставляет только нижние 55 бит — это и есть номер физической
страницы.
    // Биты 55–62 — флаги (soft-dirty, exclusive mapped и т.д.), они нам не нужны.
    uint64_t pfn = value & ((1ULL << 55) - 1);

    return pfn;
}


// Печатает виртуальные адреса, физические PFN и значения двух переменных.
// label — строка-заголовок ("До fork", "После записи" и т.д.)
// g — указатель на global_var, l — указатель на local_var
void print_ptrs(const char* label, int* g, int* l) {

    printf("--- %s ---\n", label);

    // Приводим указатели к uintptr_t (целое число = числовой адрес)
    // и передаём в get_page, чтобы узнать физические номера страниц
    uintptr_t phys_g = get_page((uintptr_t)g);
    uintptr_t phys_l = get_page((uintptr_t)l);
```

```
    // %p — формат для указателей, печатает виртуальный адрес в hex
    printf("Virt: G=%p, L=%p\n", (void*)g, (void*)l);

    // %lx — unsigned long в hex. Это и есть PFN — номер физической страницы.
    // Если до COW: у родителя и дочернего эти числа совпадут.
    // После записи в дочернем: PFN у дочернего изменится — ядро создало копию страницы.
    printf("Phys PFN: G=0x%lx, L=0x%lx\n", phys_g, phys_l);

    // *g и *l — разыменовываем указатели, получаем сами значения переменных
    printf("Val : G=%d, L=%d\n", *g, *l);

    printf("\n");
}


int main() {

    int local_var = 10;  // локальная переменная — живёт на стеке main()
    pid_t pid;           // тип для хранения PID процесса

    // Печатаем стартовое состояние — одна страница, один процесс
    print_ptrs("До fork()", &global_var, &local_var);

    // fork() — системный вызов, создаёт дочерний процесс.
    // Возвращает разные значения в каждом процессе:
    //   в родителе → PID дочернего (положительное число)
    //   в дочернем → 0
    //   при ошибке → -1 (только в родителе, дочернего нет)
    pid = fork();

    if (pid < 0) {
        // fork не удался — например, превышен лимит процессов (ulimit -u)
        perror("fork failed");
        exit(1);
    }

    else if (pid == 0) {
        // —— ДОЧЕРНИЙ ПРОЦЕСС ——
        // pid == 0 — это условие истинно только в дочернем.
        // getpid() возвращает PID текущего процесса.

        printf("--- В дочернем процессе (PID: %d) ---\n", getpid());

        // Первый print: сразу после fork, до любых записей.
        // Copy on WRITE ещё не срабатывал — дочерний только читает страницы.
        // PFN здесь будет совпадать с родительским (та же физ. страница). ( смотри
пояснение картинки ниже)

        print_ptrs("Сразу после fork (чтение)", &global_var, &local_var);

        printf("Изменяем переменные в дочернем процессе...\n\n");

        // ЗАПИСЬ в переменную → ядро получает page fault (страница помечена read-only
для COW).
        // Ядро создаёт физическую копию страницы, дочернему выдаётся новая страница.
        // Родитель продолжает видеть оригинальную страницу со старым значением.
        global_var = 200;
        local_var  = 20;

        // Второй print: после записи.
        // PFN у дочернего теперь ДРУГОЙ — это новая физ. страница (копия).
        // Виртуальный адрес тот же, что был! Только физика изменилась.
        print_ptrs("После изменения (запись)", &global_var, &local_var);

        // exit(5) — завершаем дочерний с кодом 5.
        // Этот код родитель поймает через waitpid → WEXITSTATUS.
        exit(5);
    }

    else {
        // —— РОДИТЕЛЬСКИЙ ПРОЦЕСС ——
        // pid > 0 — это PID дочернего процесса
```

```
        // Ждём 2 секунды — грубый способ дать дочернему время отработать и записать
переменные.
        // В реальном коде используют pipe/сигналы для синхронизации.
        sleep(2);

        printf("--- В родительском процессе (PID: %d) ---\n", getpid());

        // Родитель убеждается что его данные не изменились:
        // global_var == 100, local_var == 10, PFN те же что до fork.
        // COW гарантирует изоляцию — дочерний менял свою копию, не нашу.
        print_ptrs("После того как ребенок изменил свои копии", &global_var,
&local_var);
        printf("Родитель засыпает (проверьте ps/maps)...\n");
        sleep(30);



        int status;  // сюда waitpid запишет информацию о завершении дочернего

        // waitpid(pid, &status, 0) — блокируется, пока процесс pid не завершится.
        // Без этого вызова дочерний стал бы зомби-процессом:
        // он завершился, но его запись в таблице процессов осталась бы — родитель
        // не "принял" код завершения.
        waitpid(pid, &status, 0);

        // WIFEXITED(status) — макрос, проверяет что процесс завершился через exit() (не
сигналом).
        // WEXITSTATUS(status) — извлекает код: exit(5) → напечатает 5.
        if (WIFEXITED(status)) {
            printf("\nДочерний процесс завершен. Код: %d\n", WEXITSTATUS(status));
        }
    }

    return 0;
}
```

ps -o pid,ppid,stat,comm -p 4651,4652 (ту пид родителя и ребенка (выводятся в процессе выполнения команды потом 30 секунд чтобы увидеть что зомби)

```
daria@Daria-K:~/labs/OC_labs$ ps -o pid,ppid,stat,comm -p 4651,4652
    PID    PPID STAT COMMAND
   4651    4650 S+   5.1
   4652    4651 Z+   5.1
daria@Daria-K:~/labs/OC_labs$
```

Вот видишь Z — это зомби

```
daria@Daria-K:~/labs/OC_labs$ sudo /home/daria/labs/OC_labs/
5_part/5.1/5.1
[sudo: authenticate] Password:

--- До fork() ---
Virt: G=0xb543ef270010, L=0xffffd60873ac //
Phys PFN: G=0x991d2, L=0x78f07
Val : G=100, L=10

--- В дочернем процессе (PID: 3896) ---
--- Сразу после fork (чтение) ---
Virt: G=0xb543ef270010, L=0xffffd60873ac
Phys PFN: G=0x991d2, L=0x78f07
Val : G=100, L=10
```

// Родитель и ребенок имеет одинаковую виртуальную и физическую память Copt on Write то есть до того как будет первое изменение они используют одну память (чтобы выделять память только когда реально надо)

```
Изменяем переменные в дочернем процессе...

--- После изменения (запись) ---
Virt: G=0xb543ef270010, L=0xffffd60873ac
Phys PFN: G=0xcd0bd, L=0x78f07
Val : G=200, L=20

--- В родительском процессе (PID: 3895) ---
--- После того как ребенок изменил свои копии ---
Virt: G=0xb543ef270010, L=0xffffd60873ac //НЕ поменялось
Phys PFN: G=0x991d2, L=0x4a346 //Поменялось
Val : G=100, L=10
```

// ТО есть мы попытались что-то изменить и только после этого у на свиртуальная память ребенка начала отображаться не в то же место что и родитель

```
Дочерний процесс завершен. Код: 5
daria@Daria-K:~/labs/OC_labs$
```

До записи — оба процесса смотрят на одну страницу
Родитель
virt 0x601020 → ?
page table: read-only
COW флаг у обоих
Физ. страница PFN 0x1b7
global_var = 100, только чтение
Дочерний
virt 0x601020 → ?
После записи дочерним — ядро создаёт копию страницы
Родитель
virt 0x601020
PFN 0x1b7 (оригинал)
global_var = 100
Дочерний
virt 0x601020
PFN 0x3f9 (новая копия)
global_var = 200
одинаковый виртуальный адрес
разные физические страницы!